

LEONARDO LIMA DIONIZIO

TCC LEONARDO DIONIZIO

(versão pré-defesa, compilada em 3 de novembro de 2023)

Trabalho apresentado como requisito parcial à conclusão do Curso de Bacharelado em Ciência da Computação, Setor de Ciências Exatas, da Universidade Federal do Paraná.

Área de concentração: *Ciência da Computação*.

Orientador: Eduardo J. Spinosa.

CURITIBA PR

2023

RESUMO

O resumo deve conter no máximo 500 palavras, devendo ser justificado na largura da página e escrito em um único parágrafo¹ com um afastamento de 1,27 cm na primeira linha. O espaçamento entre linhas deve ser de 1,5 linhas. O resumo deve ser informativo, ou seja, é a condensação do conteúdo e expõe finalidades, metodologia, resultados e conclusões.

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetur odio sem sed wisi.

Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetur eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

Etiam euismod. Fusce facilisis lacinia dui. Suspendisse potenti. In mi erat, cursus id, nonummy sed, ullamcorper eget, sapien. Praesent pretium, magna in eleifend egestas, pede pede pretium lorem, quis consectetur tortor sapien facilisis magna. Mauris quis magna varius nulla scelerisque imperdiet. Aliquam non quam. Aliquam porttitor quam a lacus. Praesent vel arcu ut tortor cursus volutpat. In vitae pede quis diam bibendum placerat. Fusce elementum convallis neque. Sed dolor orci, scelerisque ac, dapibus nec, ultricies ut, mi. Duis nec dui quis leo sagittis commodo.

Aliquam lectus. Vivamus leo. Quisque ornare tellus ullamcorper nulla. Mauris porttitor pharetra tortor. Sed fringilla justo sed mauris. Mauris tellus. Sed non leo. Nullam elementum, magna in cursus sodales, augue est scelerisque sapien, venenatis congue nulla arcu et pede. Ut suscipit enim vel sapien. Donec congue. Maecenas urna mi, suscipit in, placerat ut, vestibulum ut, massa. Fusce ultrices nulla et nisl.

Palavras-chave: Palavra-chave 1. Palavra-chave 2. Palavra-chave 3.

¹E também não deve ter notas de rodapé; em outras palavras, não siga este exemplo... ;-)

ABSTRACT

The abstract should be the English translation of the “resumo”, no more, no less.

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetur odio sem sed wisi.

Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetur eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

Etiam euismod. Fusce facilisis lacinia dui. Suspendisse potenti. In mi erat, cursus id, nonummy sed, ullamcorper eget, sapien. Praesent pretium, magna in eleifend egestas, pede pede pretium lorem, quis consectetur tortor sapien facilisis magna. Mauris quis magna varius nulla scelerisque imperdiet. Aliquam non quam. Aliquam porttitor quam a lacus. Praesent vel arcu ut tortor cursus volutpat. In vitae pede quis diam bibendum placerat. Fusce elementum convallis neque. Sed dolor orci, scelerisque ac, dapibus nec, ultricies ut, mi. Duis nec dui quis leo sagittis commodo.

Aliquam lectus. Vivamus leo. Quisque ornare tellus ullamcorper nulla. Mauris porttitor pharetra tortor. Sed fringilla justo sed mauris. Mauris tellus. Sed non leo. Nullam elementum, magna in cursus sodales, augue est scelerisque sapien, venenatis congue nulla arcu et pede. Ut suscipit enim vel sapien. Donec congue. Maecenas urna mi, suscipit in, placerat ut, vestibulum ut, massa. Fusce ultrices nulla et nisl.

Keywords: Keyword 1. Keyword 2. Keyword 3.

LISTA DE FIGURAS

LISTA DE TABELAS

LISTA DE ACRÔNIMOS

DINF	Departamento de Informática
PPGINF	Programa de Pós-Graduação em Informática
UFPR	Universidade Federal do Paraná

LISTA DE SÍMBOLOS

α	alfa, primeira letra do alfabeto grego
β	beta, segunda letra do alfabeto grego
γ	gama, terceira letra do alfabeto grego
ω	ômega, última letra do alfabeto grego
π	pi
τ	Tempo de resposta do sistema
θ	Ângulo de incidência do raio luminoso

SUMÁRIO

1	INTRODUÇÃO	9
2	FUNDAMENTAÇÃO TEÓRICA.	12
2.1	REDES NEURAIS ADVERSÁRIAS GENERATIVAS.	12
2.2	CMA-ES	12
2.3	EVOLUÇÃO DE VARIÁVEL LATENTE (LVE)	13
2.4	GERAÇÃO DE CONTEÚDO PROCEDURAL USANDO APRENDIZADO DE MÁQUINA (PCGML)	14
2.5	MARIOGAN	14
	REFERÊNCIAS	16

1 INTRODUÇÃO

No início da adesão aos modelos neurais, sejam eles profundos ou perceptrons, tais modelos se popularizaram inicialmente nas tarefas de regressão e classificação, gerando competições populares como a ImageNet, competição em que o intuito é classificar imagens corretamente dentre 1000 classes. Ainda assim, aproximar uma distribuição só tornou-se possível com o surgimento das Redes Neurais Adversariais (GANs), inicialmente propostas em (Goodfellow et al., 2014). O paradigma do aprendizado adversarial permitiu, através do uso de uma rede geradora e uma rede discriminadora, que uma rede neural profunda se aproxime de uma distribuição de forma a, ao invés de tomar uma decisão em relação a sua entrada, gerar conteúdo que se assemelhe a um conjunto conhecido.

Em (Goodfellow et al., 2014), o intuito desta geração de conteúdo era a geração de imagens. Com o refinamento, este modelo tornou-se popular em outras tarefas, como a transferência de estilo proposta em (Zhu et al., 2020) e a tradução de texto para imagem ilustrada em (Reed et al., 2016). Embora as redes adversariais generativas tenham tornado-se bastante conhecida nestas tarefas relacionadas com imagens, suas aplicações vão muito além destas. Uma das indústrias que muito se beneficia da geração de conteúdo é o ramo dos videojogos. Embora a geração de conteúdo em jogos esteja presente desde o jogo de 1984 *Elite*, ela é muito utilizada para a geração de níveis.

Grandes exemplos do uso de geração procedural são os jogos *Minecraft*, onde cada mapa é gerado em cima de um número aleatório, uma seed, tornando cada mundo diferente do outro - de forma tão diversificada que seria impensável um humano fazer isso -, *No Man's Sky* e o recente *Starfield*, onde planetas e biomas são gerados proceduralmente num ecossistema online multijogador, e jogos do gênero rogue, onde cada vez que o jogador morre pode melhorar sua personagem e mais uma vez enfrentar os desafios do início, porém, diferentes a cada jogada, como *Rogue Legacy* (1 e 2), *Enter The Gungeon* e *Hades*.

A área acadêmica também estuda a geração de conteúdo para jogos, muitos destes estudos ligados a geração de níveis através do aprendizado adversarial, conforme levantado em (Liu et al., 2020) na **Figura X**. Um popular modelo para geração de níveis para o jogo *Super Mario Bros.* foi proposto em (Volz et al., 2018). A grande inovação do artigo que o tornou popular no nicho não foi a arquitetura da GAN em si, mas sim o fato de utilizar o conceito de evolução de variável latente (LVE) para manipular as entradas do gerador de forma que o conteúdo produzido pelo modelo seguisse certo padrão através de um algoritmo genético, o CMA-ES, proposto inicialmente em (Hansen et al., 2003). A demonstração do artigo foi treinando a rede neural proposta apenas com uma única fase e avaliando a qualidade da fase com a função objetivo do algoritmo genético e também através de métricas de um agente A estrela jogando o nível gerado, porém não se preocupa em verificar se estas fases seguem ou não de forma suficiente o

conjunto de fases conhecidas. Outra limitação de (Volz et al., 2018) é o uso apenas do CMA-ES, não se preocupando em outras formas de explorar o espaço latente, ou tentar uma abordagem multi-objetivo, algo sugerido no final do artigo como possível estudo futuro.

Este modelo foi proposto com o intuito de padronizar e simplificar as monografias, dissertações e teses produzidas no Departamento de Informática da UFPR. Ele foi vagamente inspirado nas normas da ABNT (conforme indicado em (UFPR, 2015)), mas não as segue *ipsis litteris*. Várias alterações foram feitas com o objetivo de melhorar sua estética e tornar o texto mais legível para trabalhos na área de informática. A versão atualizada deste modelo está disponível em (Maziero, 2015).

Este modelo está baseado em uma classe específica `ppginf.cls`, que aceita várias opções de compilação. A versão do documento pode ser:

- `defesa`: é gerado um documento em espaço 1,5, frente simples e sem as páginas iniciais adicionais; é uma versão adequada para receber as anotações dos membros da banca de defesa.
- `final`: é gerado um documento em espaço simples, frente/verso, com páginas iniciais (capa, ficha catalográfica, folha de aprovação, agradecimentos, etc). É uma versão bem mais compacta, mais ecológica e ideal para a impressão definitiva.

Para obter os melhores resultados, compile este modelo usando a seguinte sequência de passos:

```
pdflatex main          // compilação inicial
bibtex main            // processa referências bibliográficas
pdflatex main          // compilação final
```

ou

```
make                  // faz tudo...
```

Os principais itens considerados na formatação deste documento foram:

- Papel em formato A4, com margens de 20 mm à direita e embaixo, 30 mm nos demais lados. Não devem ser usados cabeçalhos ou rodapés além dos que estão aqui propostos.
- O texto principal do documento escrito em 12 pontos. O fonte principal do texto pode ser selecionado no arquivo `packages.tex`.
- Código-fonte, listagens e textos similares são formatados em fonte Courier 12 ou 10 pontos.
- O espaçamento padrão entre linhas é 1,5 linhas (1 linha na versão final). Não inserir espaços adicionais entre parágrafos normais. Figuras, tabelas, listagens e listas de itens devem ter um espaço adicional antes e após os mesmos.

- As páginas iniciais não são numeradas.
- O corpo do texto é numerado com algarismos arábicos (1, 2, 3, ...) a partir da introdução, até o final do documento. Os números de página devem estar situados no alto à direita (páginas direitas) ou à esquerda (páginas esquerdas).
- Expressões em inglês, grego, latim ou outras línguas devem ser enfatizadas em itálico, como *sui generis* ou *scheduling* (use o comando `\emph{ . . . }`).
- Para reforçar algo, deve-se usar somente **negrito**. Sublinhado ou MAIÚSCULAS não devem ser usados como forma de ênfase!
- As notas de rodapé também têm um modelo¹. Notas de rodapé servem para fazer algum comentário paralelo; não as use para colocar URLs, referências bibliográficas ou significado de siglas.

Felizmente o $\text{\LaTeX}$ resolve a maior parte dessas questões!

¹As notas de rodapé devem ser escritas em tamanho 10 pt, numeradas em arábico.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo revisita os tópicos fundamentais deste trabalho, incluindo os princípios de redes neurais adversariais, evolução de variável latente e algoritmos genéticos para exploração do espaço latente com o exemplo do CMA-ES; explora a literatura no tópico de geração de conteúdo procedural utilizando aprendizado de máquina; e apresenta o principal artigo que guiou este trabalho, que utiliza uma DCGAN para gerar níveis do jogo Super Mario Bros.(1985).

2.1 REDES NEURAS ADVERSARIAIS GENERATIVAS

O paradigma do Aprendizado Adversarial e As Redes Neurais Adversariais Generativas (GANs) foram introduzidos por Goodfellow et al. em 2014 como solução para o problema de estimação de máxima verossimilhança, dilema no qual o custo computacional e a complexidade são extremamente altos, podendo chegar a ser computacionalmente inviável a depender do contexto. Enquanto a gama mais popular de modelos profundos na época de publicação do artigo consistiam de apenas uma rede neural de alta profundidade usada em classificação e, em certos casos, regressão, o aprendizado adversarial proposto pelo artigo usufrui de duas MLPs.

Um desses modelos é apelidado de gerador, abreviado como G , enquanto o outro modelo é chamado de discriminador e abreviado como D . O modelo gerador representa uma função diferenciável responsável por aproximar a distribuição de seus dados de treinamento. Para gerar novos dados, a rede G projeta entradas ruidosas no espaço dos dados conhecidos, na intenção de aproximar o conteúdo gerado da distribuição dos conjuntos conhecidos.

Por sua vez, a MLP D tem como objetivo traçar um plano que separe as distribuições do conteúdo original das distribuições das projeções de G . A saída da rede discriminadora, no artigo, é descrita como um único escalar que representa a probabilidade de uma entrada não ser gerada. Essencialmente, G e D tornam-se antagônicas a partir do momento que suas respectivas funções de perda colocam como objetivo penalizar a outra. Para que isso seja possível, G é treinada para minimizar $\log(1 - D(G(z)))$ enquanto D é treinada para maximizar a probabilidade de acertar se uma entrada é sintética ou real. Este treinamento antagônico leva essas duas redes a disputarem um jogo MinMax de dois jogadores. Alternando a otimização de D e G , a rede discriminadora fica mais perto da solução ótima, enquanto a MLP geradora muda aos poucos para sua distribuição aproximar cada vez mais da original e, por consequência, tornar a solução que o discriminador busca mais complexa de se aproximar.

2.2 CMA-ES

CMA-ES, a *Covariance Matrix Adaptation Evolution Strategy*, foi instanciada primeiro em (Hansen et al., 2003). É uma estratégia evolucionária que se comporta bem com vetores de

números reais, que neste casos são as entradas do gerador, e sua principal característica é o uso de uma matriz de covariância estimada iterativamente por diferenças finitas.

Como está no seu nome, o algoritmo faz parte de um grupo de algoritmos evolucionários chamados *evolutionary strategies*, que é um grupo de algoritmos caixa preta caracterizados também pelo uso da ordenação por ranking. A CMA-ES tem se mostrado eficiente para otimizar problemas não-convexos e não-lineares em domínios contínuos.

O que este algoritmo faz é calcular uma gaussiana com base na matriz de covariância estimada. Esta gaussiana é traduzida numa elipse que se desloca de forma a contemplar as melhores soluções. Quanto mais distantes as melhores soluções estão do domínio dessa elipse, maior o *step size* dela, ou seja, seu raio. De certa forma, no começo, o *step size* costuma ser mais alto pois as melhores soluções conhecidas estão esparsas e a partir deste ponto de partida há a exploração global. Conforme a elipse fica melhor posicionada no domínio, as melhores soluções ficarão menos esparsas, fazendo com que o *step size* diminua proporcionalmente com a concentração da população.

Segue um pseudo-código do que seria o CMA-ES:

Algoritmo 1 CMA-ES

Inicialize a população: λ Inicialize o step size: σ Defina o tamanho da população: m defina o número de melhores candidatos μ Defina os pesos: $W_1, W_2, \dots, W_m$ **repeat** $x \in \lambda$ Calcula a gaussiana para x Ordena λ pelo fitness $c \leftarrow \text{calcula}_{covariância}(\lambda)$ $m \leftarrow \sum_{i=1}^{\mu} W_i \cdot X_i$ $\sigma \leftarrow \text{atualiza}(\sigma)$ **until** número de iterações $> \text{max_iter}$

2.3 EVOLUÇÃO DE VARIÁVEL LATENTE (LVE)

LVE - *Latent Variable Evolution* - surgiu primeiro para buscar digitais específicas num certo espaço mapeado por uma GAN em (Bontrager et al., 2017). Como dito anteriormente, a rede neural geradora de uma GAN mapeia entradas ruidosas para um espaço. Estas entradas ruidosas também são chamadas de variáveis latentes, ou seja, variáveis que são difíceis ou impossível de controlar e interpretar antes de um resultado concreto.

A partir disso, a técnica de LVE tem como ideia manipular as entradas ruidosas de um modelo para que sua saída se encaixe dentro de certas especificações, ou seja, adiciona ao conteúdo gerado restrições que até o momento atual não são controláveis a nível de treinamento, pois uma rede geradora tem como única função reproduzir a distribuição de seus dados de teste. Em (Bontrager et al., 2017), a técnica de LVE é utilizada para a quebra de identificação de digitais. A técnica consiste de duas etapas: primeiro, treinar uma rede neural para gerar o conteúdo alvo e, após isso, usar um algoritmo para buscar no espaço latente uma entrada que gere uma saída dentro das condições especificadas.

O algoritmo utilizado por (Bontrager et al., 2017) para busca no espaço latente é o já explicado anteriormente CMA-ES. A justificativa do uso do CMA-ES pelo autores é que a função

objetivo é não contínua e, pela natureza de redes convolucionais, também não é linearmente separável.

2.4 GERAÇÃO DE CONTEÚDO PROCEDURAL USANDO APRENDIZADO DE MÁQUINA (PCGML)

A survey de Summerville e colegas (Summerville et al., 2018) define *Procedural Content Generation Via Machine Learning* como "geração de conteúdo de jogos usando modelos de *machine learning* em conteúdos existentes". Na área, busca-se aumentar o valor de replay, salvar memória, reduzir o custo e o esforço de produção, diversificar estética e reduzir a presença humana na geração de conteúdo de jogos. Na área acadêmica, além disso, pode-se dizer que se explora novas maneiras de experienciar os jogos, além de ser um desafio que estimula a criatividade computacional e o entendimento de *game design* pela construção de modelos.

Alguns exemplos de PCGML são o *Bardo Composer* (Ferreira et al., 2020) que gera músicas para RPGs de Mesa, geração de personagens em jogos no estilo pixel-art em (Rebouças Serpa e Formico Rodrigues, 2019), ou até mesmo aplicações mais práticas, como a desenvolvedora *CD Projekt RED*, que fez uma parceria com a empresa canadense *Jali Research* para, através de técnicas híbridas de inteligência artificial clássica e aprendizado de máquina, fazer com que a boca dos personagens se adapte a dublagem correspondente do diálogo, e o recente anúncio da desenvolvedora francesa *Ubisoft* do desenvolvimento de uma ferramenta para ajuda de construção de narrativas de jogos, a *Ubisoft Ghostwriter*.

Porém, a geração de conteúdo procedural data ao jogo de 1984 *Elite*, onde os desenvolvedores usavam uma sequência definida de sementes aleatórias para gerar composições de planetas na intenção de economizar memória armazenando apenas um gerador ao invés dos planetas inteiros.

Como observado em (Liu et al., 2020), a grande parte dos conteúdos ainda é focado em geração procedural de níveis, e este tópico é ainda dominado pelo uso de GANs.

2.5 MARIOGAN

Em maio de 2018 publicou-se o artigo que deu origem ao que outros entusiastas do estudo da PCGML chamam de MarioGAN (Volz et al., 2018). O artigo aborda o uso de GANs, mas não foca na arquitetura da rede geradora ou discriminadora, mas sim utiliza a ideia de LVE para geração de níveis procedurais no jogo Super Mario Bros. Aqui, seguindo a ideia de (Bontrager et al., 2017), o CMA-ES é utilizado para adicionar alguma restrição ao espaço latente do gerador, sendo testadas quatro funções-objetivo diferentes, duas delas relacionadas a disposição dos blocos da fase e outras duas em relação ao teste das fases pelo agente A* vencedor da competição Mario AI de 2009 (Togelius et al., 2010) de autoria de Robin Baumgarten.

Para representar as fases, os autores utilizaram a representação do jogo Super Mario Bros. disponível no repositório do *github* aberto da Video Game Level Corpus (VGLC) (Summerville

et al., 2016), que, entre suas particularidades, simplificam a representação do jogo ao ignorar níveis submersos, onde a física do jogo diverge do comum, e ao representar todos os inimigos da mesma maneira, tornando a tarefa possivelmente mais simples. O gerador utilizado no artigo é treinado apenas com o primeiro nível do jogo, na intenção de mostrar como a LVE e a GAN que treinaram poderiam gerar níveis diferentes e jogáveis. Outra escolha interessante do artigo da MarioGAN (Volz et al., 2018) é que, para gerar múltiplas amostras com apenas uma fase, a representação foi separada com base em uma janela deslizante com dimensões de 14 de altura e 28 de largura 28, com passo de um bloco.

REFERÊNCIAS

- Bontrager, P., Togelius, J. e Memon, N. (2017). Deepmasterprint: Generating fingerprints for presentation attacks.
- Ferreira, L. N., Lelis, L. H. S. e Whitehead, J. (2020). Computer-generated music for tabletop role-playing games.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. e Bengio, Y. (2014). Generative adversarial networks. *Advances in Neural Information Processing Systems*, 3.
- Hansen, N., Müller, S. D. e Koumoutsakos, P. (2003). Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (cma-es). *Evolutionary Computation*, 11(1):1–18.
- Liu, J., Snodgrass, S., Khalifa, A., Risi, S., Yannakakis, G. N. e Togelius, J. (2020). Deep learning for procedural content generation. *Neural Computing and Applications*, 33(1):19–37.
- Maziero, C. (2015). Modelo PPGInf UFPR para teses e dissertações. <http://www.inf.ufpr.br/maziero>. Acessado em 30/11/2015.
- Rebouças Serpa, Y. e Formico Rodrigues, M. A. (2019). Towards machine-learning assisted asset generation for games: A study on pixel art sprite sheets. Em *2019 18th Brazilian Symposium on Computer Games and Digital Entertainment (SBGames)*, páginas 182–191.
- Reed, S., Akata, Z., Yan, X., Logeswaran, L., Schiele, B. e Lee, H. (2016). Generative adversarial text to image synthesis.
- Summerville, A., Snodgrass, S., Guzdial, M., Holmgård, C., Hoover, A. K., Isaksen, A., Nealen, A. e Togelius, J. (2018). Procedural content generation via machine learning (pcgml).
- Summerville, A. J., Snodgrass, S., Mateas, M. e n'on Villar, S. O. (2016). The vglc: The video game level corpus. *Proceedings of the 7th Workshop on Procedural Content Generation*.
- Togelius, J., Karakovskiy, S. e Baumgarten, R. (2010). The 2009 mario ai competition. páginas 1 – 8.
- UFPR, B. (2015). Manual de normalização de documentos científicos de acordo com as normas da ABNT. Relatório Técnico ISBN 9788584800025, Sistema de Bibliotecas – Universidade Federal do Paraná, Curitiba PR.

- Volz, V., Schrum, J., Liu, J., Lucas, S. M., Smith, A. M. e Risi, S. (2018). Evolving mario levels in the latent space of a deep convolutional generative adversarial network. Em *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 2018)*, New York, NY, USA. ACM.
- Zhu, J.-Y., Park, T., Isola, P. e Efros, A. A. (2020). Unpaired image-to-image translation using cycle-consistent adversarial networks.